

10.3.1 Reduction to tridiagonal form ¶

10 3 1 Reduction to tridiagonal form

fit width



In this section, we see that if $A^{(0)}$ is a tridiagonal matrix, then so are all $A^{(k)}$. This reduces the cost of each iteration of the QR algorithm from $O(m^3)$ flops to $O(m)$ flops if only the eigenvalues are computed and $O(m^2)$ flops if the eigenvectors are also desired. Thus, if matrix A is first reduced to a tridiagonal matrix via (unitary) similarity transformations, then the cost of finding its eigenvalues and eigenvectors is reduced from $O(m^4)$ to $O(m^3)$ flops. Fortunately, there is an algorithm for reducing a matrix to tridiagonal form that requires $O(m^3)$ operations.

The basic algorithm for reducing a Hermitian matrix to tridiagonal form, overwriting the original matrix with the result, can be explained as follows. We assume that A is stored only in the lower triangular part of the matrix and that only the diagonal and subdiagonal of the tridiagonal matrix is computed, overwriting those parts of A . Finally, the Householder vectors used to zero out parts of A can overwrite the entries that they annihilate (set to zero), much like we did when computing the Householder QR factorization.

Recall that in [Subsubsection 3.3.3.3](#), we introduced the function

$$\left[\begin{pmatrix} \rho \\ u_2 \end{pmatrix}, \tau\right] := \text{Housev}\left(\begin{pmatrix} \chi_1 \\ x_2 \end{pmatrix}\right)$$

to compute the vector $u = \begin{pmatrix} 1 \\ u_2 \end{pmatrix}$ that reflects x into $\begin{bmatrix} \pm \\ \end{bmatrix} \|x\|_2 e_0$ so that

$$\left(I - \frac{1}{\tau} \begin{pmatrix} 1 \\ u_2 \end{pmatrix} \begin{pmatrix} 1 \\ u_2 \end{pmatrix}^H\right) \begin{pmatrix} \chi_1 \\ x_2 \end{pmatrix} = \begin{bmatrix} \pm \\ \end{bmatrix} \underbrace{n \|x\|_2}_{\rho} e_0.$$

We are going to use a variation on this function:

$$[u, \tau] := \text{Housev}(x)$$

implemented by the function

function [u, tau] = Housev1(x)

We also reintroduce the notation $H(x)$ for the transformation $I - \frac{1}{\tau}uu^H$ where u and τ are computed by $\text{Housev1}(x)$.

We now describe an algorithm for reducing a Hermitian matrix to tridiagonal form:

- Partition $A \rightarrow \begin{pmatrix} \alpha_{11} & \star \\ a_{21} & A_{22} \end{pmatrix}$. Here the $\star$ denotes a part of a matrix that is neither stored nor updated.
- Update $[a_{21}, \tau] := \text{Housev1}(a_{21})$. This overwrites the first element of a_{21} with $\begin{bmatrix} \pm \\ \end{bmatrix} \|a_{21}\|_2$ and the remainder with all but the first element of the Householder vector u . Implicitly, the elements below the first element equal zero in the updated matrix A .
- Update

$$A_{22} := H(a_{21})A_{22}H(a_{21}).$$

Since A_{22} is Hermitian both before and after the update, only the lower triangular part of the matrix needs to be updated.

- Continue this process with the updated A_{22} .

This approach is illustrated in [Figure 10.3.1.1](#).

x	x	x	x	x
x	x	x	x	x
x	x	x	x	x
x	x	x	x	x
x	x	x	x	x

Original matrix

$\longrightarrow$

x	x	0	0	0
x	x	x	x	x
0	x	x	x	x
0	x	x	x	x
0	x	x	x	x

First iteration

$\longrightarrow$

x	x	0	0	0
x	x	x	0	0
0	x	x	x	x
0	0	x	x	x
0	0	x	x	x

Second iteration

$$\begin{array}{c|ccc} \times & \times & & \\ \times & \times & & \\ \hline 0 & \times & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & 0 & \times \end{array} \quad \begin{array}{c|ccc} \times & \times & 0 & \\ \times & \times & \times & \\ \hline 0 & \times & \times & \times \\ 0 & 0 & \times & \times \\ 0 & 0 & 0 & \times \end{array}$$

Third iteration

Figure 10.3.1.1. An illustration of the reduction of a Hermitian matrix to tridiagonal form. The $\times$ s denote nonzero elements in the matrix. The gray entries above the diagonal are not actually updated.

The update of A_{22} warrants closer scrutiny:

$$\begin{aligned} A_{22} &:= H(a_{21})A_{22}H(a_{21}) \\ &= \left(I - \frac{1}{\tau}u_{21}u_{21}^H\right)A_{22}\left(I - \frac{1}{\tau}u_{21}u_{21}^H\right) \\ &= \underbrace{\left(A_{22} - \frac{1}{\tau}u_{21}u_{21}^HA_{22}\right)}_{y_{21}^H}\left(I - \frac{1}{\tau}u_{21}u_{21}^H\right) \\ &= A_{22} - \frac{1}{\tau}u_{21}y_{21}^H - \frac{1}{\tau}\underbrace{A_{22}u_{21}}_{y_{21}} + \frac{1}{\tau^2}\underbrace{y_{21}^Hy_{21}}_{2\beta}u_{21}^H \\ &= A_{22} - \left(\frac{1}{\tau}u_{21}y_{21}^H - \frac{\beta}{\tau^2}u_{21}u_{21}^H\right) - \left(\frac{1}{\tau}y_{21}u_{21}^H - \frac{\beta}{\tau^2}u_{21}u_{21}^H\right) \\ &= A_{22} - u_{21}\underbrace{\left(\frac{1}{\tau}y_{21}^H - \frac{\beta}{\tau^2}u_{21}^H\right)}_{w_{21}^H} - \underbrace{\frac{1}{\tau}\left(y_{21} - \frac{\beta}{\tau}u_{21}\right)}_{w_{21}}u_{21}^H \\ &= \underbrace{A_{22} - u_{21}w_{21}^H - w_{21}u_{21}^H}_{\text{Hermitian rank-2 update}}. \end{aligned}$$

This formulation has two advantages: it requires fewer computations and it does not generate an intermediate result that is not Hermitian. An algorithm that implements all these insights is given in [Figure 10.3.1.2](#).

```

[A, t] := TriRed-unb(A, t)

A → ( (ATL | ATR) / (ABL | ABR) ), t → ( tT / tB )

ATL is 0 × 0 and tT has 0 elements
while m(ATL) < m(A) - 2
    ( (ATL | ATR) / (ABL | ABR) ) → ( (A00 | a01 A02) / (a10T | α11 a12T) / (A20 | a21 A22) ), ( tT / tB ) → ( t0 / τ1 / t2 )

    [a21, τ1] := House1(a21)
    u21 = a21 with first element replaced with 1
    Update A22 := H(u21)A22H(u21) via the steps
        {
            y21 := A22u21 (Hermitian matrix-vector multiply!)
            β := u21Hy21/2
            w21 := (y21 - βu21/τ1)/τ1
            A22 := A22 - tril(u21w21H + w21u21H) (Hermitian-rank-2-update)
        }

    ( (ATL | ATR) / (ABL | ABR) ) ← ( (A00 | a01 A02) / (a10T | α11 a12T) / (A20 | a21 A22) ), ( tT / tB ) ← ( t0 / τ1 / t2 )

endwhile

```

Figure 10.3.1.2. Basic algorithm for reduction of a Hermitian matrix to tridiagonal form.

During the first iteration, when updating $(m-1) \times (m-1)$ matrix A_{22} , the bulk of computation is in the computation of $y_{21} := A_{22}u_{21}$, at $2(m-1)^2$ flops, and $A_{22} := A_{22} - (u_{21}w_{21}^H + w_{21}u_{21}^H)$, at $2(m-1)^2$ flops. The total cost for reducing $m \times m$ matrix A to tridiagonal form is therefore approximately

$$\sum_{k=0}^{m-1} 4(m-k-1)^2 \text{ flops.}$$

By substituting $j = m - k - 1$ we find that

$$\sum_{k=0}^{m-1} 4(m-k-1)^2 \text{ flops} = 4 \sum_{j=0}^{m-1} j^2 \text{ flops} \approx 4 \int_0^m x^2 dx = \frac{4}{3}m^3 \text{ flops.}$$

This equals, approximately, the cost of one QR factorization of matrix A .

Homework 10.3.1.1. A more straight forward way of updating A_{22} is given by

$$\begin{aligned} A_{22} &:= \left(I - \frac{1}{\tau}u_{21}u_{21}^H\right)A_{22}\left(I - \frac{1}{\tau}u_{21}u_{21}^H\right) \\ &= \underbrace{\left(A_{22} - \frac{1}{\tau}u_{21}u_{21}^HA_{22}\right)}_{B_{22}}\left(I - \frac{1}{\tau}u_{21}u_{21}^H\right) \\ &= B_{22} - \frac{1}{\tau}\underbrace{B_{22}u_{21}}_{x_{21}}u_{21}^H. \end{aligned}$$

This suggests the steps

- Compute $y_{21} = A_{22}u_{21}$. (Hermitian matrix-vector multiplication).
- Compute $B_{22} = A_{22} - \frac{1}{\tau}u_{21}y_{21}^H$. (Rank-1 update yielding a *nonHermitian* intermediate matrix).
- Compute $x_{21} = B_{22}u_{21}$. (Matrix-vector multiplication).

- Compute $A_{22} = B_{22} - \frac{1}{\tau} x_{21} u_{21}^H$. (Rank-1 update yielding a *Hermitian* final matrix).

Estimate the cost of this alternative approach. What other disadvantage(s) does this approach have?

▼ [Solution](#)

During the k th iteration, for $k = 0, 1, \dots, m-1$ the costs for the various steps are as follows:

- Compute $y_{21} = A_{22} u_{21}$. (Hermitian matrix-vector multiplication). Cost: approximately $2(m-1)^2$ flops.
- Compute $B_{22} = A_{22} - \frac{1}{\tau} u_{21} y_{21}^H$. (Rank-1 update yielding a *nonHermitian* intermediate matrix). Cost: approximately $2(m-1)^2$ flops since the intermediate matrix B_{22} is not Hermitian.
- Compute $x_{21} = B_{22} u_{21}$. (Matrix-vector multiplication). Cost: approximately $2(m-1)^2$ flops.
- Compute $A_{22} = B_{22} - \frac{1}{\tau} x_{21} y_{21}^H$. Only the lower triangular part of A_{22} needs to be computed. Cost: approximately $(m-1)^2$ flops.

Thus, the total cost per iteration is, approximately

$$7(m-1)^2 \text{ flops.}$$

The total cost is then, approximately,

$$\sum_{k=0}^{m-1} 7(m-k-1)^2 \text{ flops} = 7 \sum_{j=0}^{m-1} j^2 \text{ flops} \approx 7 \int_0^m x^2 dx = \frac{7}{3} m^3 \text{ flops.}$$

This almost doubles the cost of the reduction to tridiagonal form.

An additional disadvantage is that a nonsquare intermediate matrix must be stored.

The diagonal elements of a Hermitian matrix are real. Hence the tridiagonal matrix has real values on its diagonal. A post process (that follows the reduction to tridiagonal form) can be used to convert the elements of the subdiagonal to real values as well. The advantage of this is that in the subsequent computation, that computes the eigenvalues of the tridiagonal matrix and accumulates the eigenvectors, only needs to perform real (floating point) arithmetic.

Ponder This 10.3.1.2. Propose a postprocess that converts the off-diagonal elements of a tridiagonal Hermitian matrix to real values. The postprocess must be equivalent to applying a unitary similarity transformation so that eigenvalues are preserved.

n You may want to start by looking at

$$A = \begin{pmatrix} \alpha_{0,0} & \bar{\alpha}_{1,0} \\ \alpha_{1,0} & \alpha_{1,1} \end{pmatrix},$$

where the diagonal elements are real-valued and the off-diagonal elements are complex-valued. Then move on to

$$A = \begin{pmatrix} \alpha_{0,0} & \bar{\alpha}_{1,0} & 0 \\ \alpha_{1,0} & \alpha_{1,1} & \bar{\alpha}_{2,1} \\ 0 & \alpha_{2,1} & \alpha_{2,2} \end{pmatrix}.$$

What is the pattern?

Homework 10.3.1.3. You may want to start by executing `git pull` to update your directory `Assignments`.

In directory `Assignments/Week10/matlab/`, you will find the following files:

- `Housev1.m`: An implementation of the function `Housev1`, mentioned in the unit.
- `TriRed.m`: A code skeleton for a function that reduces a Hermitian matrix to a tridiagonal matrix. Only the lower triangular part of the input and output are stored.

```
[ T, t ] = TriRed( A, t )
```

returns the diagonal and first subdiagonal of the tridiagonal matrix in `T`, stores the Householder vectors below the first subdiagonal, and returns the scalars τ in vector `t`.

- `TriFromBi.m`: A function that takes the diagonal and first subdiagonal in the input matrix and returns the tridiagonal matrix that they define.

```
T = TriFromBi( A )
```

- `test_TriRed.m`: A script that tests `TriRed`.

With these resources, you are to complete `TriRed` by implementing the algorithm in [Figure 10.3.1.2](#).

Be sure to look at the hint!

▼ [Hint](#) ▼ [Solution](#)

- `Assignments/Week10/answers/TriRed.m`.

If `A` holds Hermitian matrix A , storing only the lower triangular part, then Ax is implemented in Matlab as

```
( tril( A ) + tril( A, -1)' ) * x;
```

Updating only the lower triangular part of array `A` with $A := A - B$ is accomplished by

```
A = A - tril( B );
```